Universite de Technologie de Belfort-Montbeliard

PROJET SR70

Gestion avancee d'un systeme d'ascenseurs

Simulation en langage C de deux ascenseurs dans un immeuble de onze niveaux

Membres Floryan BORNET
Tim CARRARA
Enzo VIDY

Annee universitaire : 2026-2027

Table des matières

1 Presentation du sujet et objectifs **2**
1.1 Perimetre retenu 2

2 Fonctionnalites realisees **2**

3 Architecture generale **3**
3.1 Organisation des modules 3

4 Choix de conception **4**
4.1 Processus pour les ascenseurs, threads pour les usagers 4
4.2 Types de messages separes 4
4.3 Heuristique d'affectation 4
4.4 File d'attente et priorites 4

5 Concepts de programmation systeme Linux **4**

6 Deroulement d'une simulation **5**

7 Tests et resultats **5**
7.1 Validation manuelle avec panne 6
7.2 Exemple de sortie terminal 6

8 Difficultes rencontrees et solutions **6**

9 Limites et ameliorations possibles **7**

10 Conclusion **7**

A Compilation et execution **8**

1 Presentation du sujet et objectifs

Le sujet consiste a developper une simulation avancee d'un systeme de gestion de deux ascenseurs pour un immeuble de bureaux de dix etages, auquel s'ajoute un niveau d'entree numerote 0. Les ascenseurs doivent repondre aux demandes des usagers, limiter les temps d'attente et communiquer au moyen de mecanismes de programmation systeme.

Le projet a deux objectifs complementaires. Le premier est fonctionnel : simuler l'arrivee des appels, la selection d'un ascenseur et les deplacements entre les niveaux 0 et 10. Le second est pedagogique : mettre en oeuvre plusieurs mecanismes Linux et etre capable de justifier leur utilisation, notamment les processus, les threads, les IPC et la synchronisation.

1.1 Perimetre retenu

Le programme fonctionne dans un terminal Linux. Les demandes peuvent etre generees automatiquement ou saisies dans une console interactive. Le nombre d'usagers, le delai entre les demandes, la graine aleatoire, la probabilite de panne, la duree de reparation et la capacite d'affectation peuvent etre configures en ligne de commande.

2 Fonctionnalites realisees

Le tableau suivant distingue les fonctionnalites effectivement implementees des idees volontairement laissees comme evolutions.

Fonctionnalite	**Implementation dans le projet**
Modelisation de l'immeuble	Structures `Immeuble` et `Etage`, avec onze niveaux numerotes de 0 a 10 et des espaces nommes : accueil, services, cafeteria, salle de reunion et direction.
Simulation de frequentation	Creation de threads POSIX. Chaque thread construit une demande aleatoire avec un depart, une destination differente et une priorite. Les creations sont espacees pour simuler un flux d'appels.
Deux ascenseurs	Deux processus fils independants crees avec `fork()`. Chaque processus execute la boucle de son ascenseur.
Gestion des appels	Un thread gestionnaire recoit les demandes, les place dans une file d'attente et les transmet aux ascenseurs par messages.
Optimisation	Heuristique de cout integrant la distance jusqu'au depart, la charge deja affectee, le trajet restant et la coherence du sens de deplacement.
Priorites	Trois niveaux : normale, prioritaire et urgente. A priorite egale, les demandes les plus anciennes sont traitees d'abord.
Communication	Files de messages System V pour les appels, les commandes et les confirmations.
Synchronisation	Semaphore POSIX partage entre processus pour proteger l'etat partage et synchroniser les evenements. Des mutex pthread protegent les donnees partagees entre threads du processus principal.
Pannes	En mode automatique, une panne peut etre tiree aleatoirement. En mode interactif, elle peut etre declenchee manuellement. Une demande interrompue est remise en attente apres une alerte.
Statistiques	Export CSV distinct par execution dans `resultats/`, avec priorite, ascenseur, depart, destination, attente, duree et nombre de tentatives.

Fonctionnalite	Implementation dans le projet
Signaux et arret	Prise en compte de `SIGINT` et `SIGTERM`, arret explicite des processus fils et liberation des ressources IPC.

Les demandes urgentes sont uniquement des categories de simulation. Le programme ne modelise pas de situation reelle ni de personne reelle. L'interface utilisateur est une interface en ligne de commande ; aucune interface graphique n'est necessaire pour demontrer les mecanismes systeme du projet.

3 Architecture generale

L'application est composee d'un processus principal, de deux processus ascenseurs, d'un thread gestionnaire et d'un thread par usager simule. Les ressources IPC sont creees avant les processus fils. Ce choix permet d'effectuer les appels a `fork()` avant de creer les threads, ce qui evite de forker un processus deja multithreade dans cette architecture.

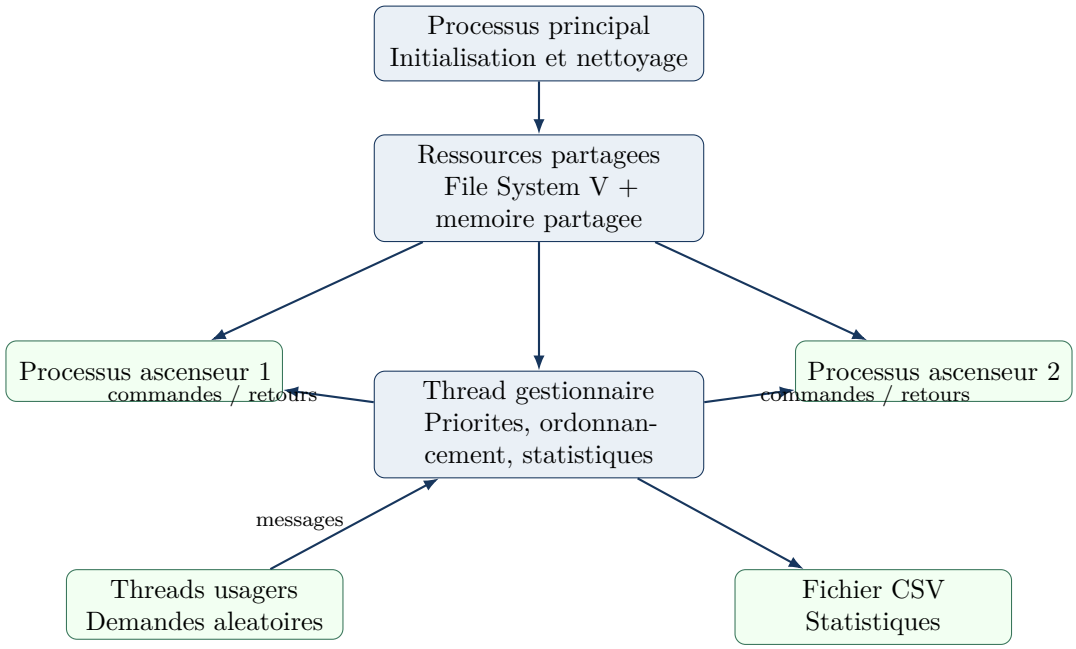


FIGURE 1 – Vue logique des processus, threads et communications.

3.1 Organisation des modules

— `main.c` : configuration, creation des ressources, creation des processus et des threads, puis nettoyage.

— `ascenseur.c` : reception des commandes, deplacement, prise en charge, panne et reprise.

— `utilisateur.c` : generation concurrente des demandes.

— `gestionnaire.c` : reception, file d'attente, affectation, gestion des confirmations et collecte des statistiques.

— `ordonnancement.c` : calcul du cout d'affectation et comparaison des priorites.

— `communication.c` : encapsulation des appels System V `msgsnd()` et `msgrcv()`.

— `synchronisation.c` : creation de la memoire partagee et des semaphores POSIX partages.

— `alertes.c`, `scenario.c`, `statistiques.c` et `immeuble.c` : alertes, pannes, export CSV et description des niveaux.

4 Choix de conception

4.1 Processus pour les ascenseurs, threads pour les usagers

Les ascenseurs sont des processus distincts afin de montrer la creation et la gestion de processus avec `fork()`. Chaque processus possede sa propre boucle de reception et de deplacement. Les usagers sont representes par des threads car ils partagent les ressources du processus principal et ne necessitent pas un espace d'adressage independant.

Le processus principal cree les deux processus fils avant de lancer les threads. Un thread gestionnaire est ensuite lance en parallele des threads usagers : les demandes peuvent donc etre traitees pendant leur generation, au lieu d'attendre que tous les usagers aient fini.

4.2 Types de messages separes

Les messages System V commencent par un champ `long type`. Le projet reserve des types distincts :

Type	Sens	Contenu
1	Usager vers gestionnaire	Demande d'appel
100 et 101	Gestionnaire vers ascenseur	Commande de trajet
200 et 201	Ascenseur vers gestionnaire	Confirmation ou echec

Cette separation evite qu'un processus ascenseur consomme une demande brute destinee au gestionnaire. Elle permet aussi au gestionnaire de recuperer le message de fin correspondant a chaque ascenseur.

4.3 Heuristique d'affectation

Le gestionnaire compare les deux ascenseurs pour chaque demande. Un ascenseur hors service est exclu. Pour les autres, le cout estime combine :

— la distance entre sa position et l'etage de depart de l'appel ;

— une penalite proportionnelle au nombre de commandes deja affectees ;

— si l'ascenseur est deja en mouvement, la distance restante jusqu'a sa destination courante ;

— la distance supplementaire entre cette destination et le nouvel appel ;

— un bonus ou une penalite selon que le sens de deplacement est coherent avec l'appel.

L'ascenseur dont le cout est le plus faible est selectionne. Cette strategie est une heuristique locale : elle ameliore la decision par rapport a la seule distance, mais ne garantit pas un optimum global. Les commandes affectees a un meme ascenseur sont executees sequentiellement.

4.4 File d'attente et priorites

La file d'attente du gestionnaire selectionne d'abord la demande de priorite la plus elevee. Si plusieurs demandes possedent la meme priorite, la plus ancienne est choisie. Les niveaux de priorite sont tires au sort lors de la generation des demandes, avec une proportion plus faible de demandes urgentes.

5 Concepts de programmation systeme Linux

Concept	Mise en oeuvre
Processus – `fork()`	Le processus principal cree deux fils. Chaque fils execute `processus_ascenseur()` et termine avec `_exit()`.
Threads – pthreads	`pthread_create()` lance un thread gestionnaire et les threads usagers. `pthread_join()` attend leur fin avant le nettoyage.
Files de messages System V	`msgget()`, `msgsnd()`, `msgrcv()` et `msgctl()` assurent le transport des appels, commandes et confirmations.
Memoire partagee System V	`shmget()` et `shmat()` creent une zone contenant la position, la direction, la destination, l'occupation et l'etat de panne des ascenseurs.
Semaphores POSIX partages	`sem_init()` initialise les semaphores avec `pshared = 1`. Ils protegent l'etat partage et reveillent le gestionnaire lors des nouveaux appels, fins de trajet et pannes.
Mutex pthread	Deux mutex protegent la generation aleatoire et les compteurs, qui sont partages uniquement entre les threads du processus principal.
Signaux	Le processus principal et les ascenseurs installent des gestionnaires pour `SIGINT` et `SIGTERM`. Le programme organise ensuite l'arret des fils et la liberation des ressources.
Fichiers	Les statistiques sont enregistrees dans un fichier CSV afin de pouvoir etre analysees apres la simulation.

Les semaphores sont initialises dans la memoire partagee avant le `fork()`. Leur parametre `pshared` vaut 1 : ils sont donc utilisables par les processus ascenseurs et le processus principal.

6 Deroulement d'une simulation

1. Le programme lit les options de lancement et cree le dossier de resultats.
2. Il cree une file de messages System V et une zone de memoire partagee contenant les etats initiaux des deux ascenseurs.
3. Il cree les deux processus ascenseurs avec `fork()`.
4. Il lance le thread gestionnaire, puis cree progressivement les threads usagers. Chaque thread genere un depart, une destination et une priorite, puis envoie une demande.
5. Le gestionnaire collecte les demandes, les trie par priorite et choisit un ascenseur selon le cout estime. Il peut affecter une demande a un ascenseur qui a deja des commandes en attente.
6. L'ascenseur rejoint l'etage de depart, signale la prise en charge, puis se rend a l'etage de destination. Il renvoie une confirmation.
7. En cas de panne simulee avant le trajet, l'ascenseur envoie un message d'echec. Le gestionnaire remet la demande dans la file ; l'ascenseur est temporairement indisponible puis revient en service apres la reparation simulee.
8. Lorsque toutes les demandes ont ete servies, le processus principal envoie un ordre d'arret aux fils, attend leur terminaison et supprime la file de messages et la memoire partagee.

7 Tests et resultats

La compilation utilise GCC avec les options de controle `-Wall`, `-Wextra` et `-Wpedantic`. Les validations presentees ci-dessous sont des executions manuelles realisees sous Linux/WSL.

7.1 Validation manuelle avec panne

Le test d'integration lance la commande suivante :

```
./gestion-ascenseurs -n 10 -s 4242 -d 1 -p 100 -r 10
```

Cette configuration augmente la probabilite d'apparition d'une panne. Elle permet de verifier qu'une demande interrompue est remise en attente puis terminee apres la reparation. Les durees et la repartition des demandes varient selon l'execution ; elles doivent etre relevees dans les captures produites par le groupe.

Capture a inserer : execution automatique avec une panne, reaffectation de la demande et bilan final.

FIGURE 2 – Exemple d'execution automatique a remplacer par une capture du groupe.

7.2 Exemple de sortie terminal

Capture a inserer : console interactive, commande `etat` et creation d'un appel prioritaire.

FIGURE 3 – Exemple de mode interactif a remplacer par une capture du groupe.

Le dossier `resultats/` contient un fichier CSV distinct par execution. Chaque fichier enregistre l'identifiant, la priorite, l'ascenseur choisi, les etages de depart et d'arrivee, l'attente avant prise en charge, la duree totale et le nombre de tentatives.

8 Difficultes rencontrees et solutions

— **Confusion possible entre les messages :** les appels et les commandes du premier ascenseur utilisaient initialement un type de message qui pouvait se chevaucher. Des plages de types distinctes ont ete reservees pour les demandes, les commandes et les retours.

— **Acces concurrents a l'etat des ascenseurs :** plusieurs processus consultent ou modifient les positions et etats. La zone de memoire partagee est maintenant protegee par un mutex partage entre processus.

Capture a inserer : contenu d'un fichier CSV dans `resultats/`.

FIGURE 4 – Export des statistiques de simulation.

— **Reveil du gestionnaire :** une interrogation continue de la file aurait genere une boucle d'attente active. Le semaphore permet de reveiller le gestionnaire lors d'un nouvel evenement, avec une attente bornee pour traiter egalement les confirmations.

— **Panne et perte d'une demande :** une panne ne doit pas faire disparaitre l'appel. L'ascenseur transmet un statut d'echec, puis le gestionnaire reintegre la demande dans la file avant de tenter une nouvelle affectation.

— **Ordre des traces :** la sortie standard est configuree en mode ligne afin de conserver des traces lisibles lorsque la sortie du programme est redirigee vers un fichier.

9 Limites et ameliorations possibles

Le projet est une simulation pedagogique, et non un logiciel de commande d'un equipement reel. Plusieurs simplifications sont assumees :

— les commandes d'un ascenseur sont traitees sequentiellement ; le programme ne construit pas un itineraire multi-arrets commun a plusieurs passagers ;

— l'algorithme d'affectation est une heuristique locale et non un solveur d'optimisation globale ;

— une panne automatique est limitee a un scenario par execution ; le mode interactif permet de provoquer des pannes manuelles ;

— la capacite represente le nombre maximal de demandes affectees a un ascenseur, et non un nombre physique de passagers ;

— l'interface reste textuelle et ne propose pas de representation graphique de l'immeuble.

Des evolutions possibles seraient l'ajout de plusieurs arrets par trajet, d'une capacite physique avec surcharge, de scenarios horaires plus riches, de pannes pendant un trajet et d'une analyse graphique du CSV.

10 Conclusion

Ce projet met en oeuvre une simulation concurrente de deux ascenseurs en langage C sous Linux. Il mobilise les processus, les threads, les files de messages System V, la memoire partagee, des semaphores POSIX partages, des mutex entre threads, les signaux et l'enregistrement de fichiers. La gestion des priorites, la prise en compte de la direction et de la charge, les pannes simulees, le mode interactif et les statistiques apportent un niveau fonctionnel allant au-dela d'un simple deplacement aleatoire.

Les executions manuelles permettent de verifier la compilation, les appels automatiques et interactifs, les pannes, les reparations, les priorites et l'export CSV. Le groupe doit personnaliser la page de garde, inserer ses propres captures et etre en mesure d'expliquer les choix techniques presentes dans ce rapport.

A Compilation et execution

```
make
make run
make help
./gestion-ascenseurs -h
./gestion-ascenseurs -n 20 -s 42 -d 1
./gestion-ascenseurs -n 20 -s 42 -d 1 -p 15 -r 10
./gestion-ascenseurs -i -n 5 -d 0 -p 0
```